
UT2 - Entornos de Desarrollo

— 1ºDAM / 1º DAW - ETS - 25/26 —

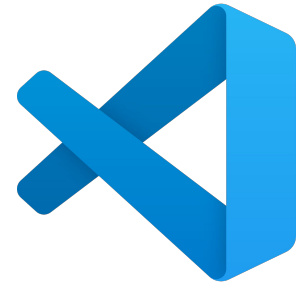
En esta unidad veremos:



1. La utilidad de los entornos de desarrollo integrados
2. Componentes de un entorno de desarrollo
3. Instalación de un entorno de desarrollo
4. Depuración de Java

Introducción

En este tema, se verán cuáles son los componentes de un IDE, se explicará la instalación de dos entornos de desarrollo (Apache NetBeans y Visual Studio Code) y cómo se pueden crear programas mediante ellos.



1. La utilidad de los entornos de desarrollo integrados

Proceso de creación de programas

El proceso de creación de programas se sintetiza en los siguientes pasos:

1. Creación del código fuente del programa → **Editor de textos**
2. Transformación del código fuente en código objeto mediante el empleo de un traductor (compilador o intérprete). → **Compilador o intérprete**
3. Obtención del código ejecutable insertando en el código objeto una serie de rutinas o librerías necesarias. → **Enlazador**

Como el proceso es siempre (más o menos) el mismo, en busca de la facilidad para el usuario (programador) se crean los **entornos de desarrollo integrados**.

La forma más habitual de referirse a los entornos de desarrollo o entornos de desarrollo integrados es empleando sus siglas en inglés **IDE** (*Integrated Development Environment*).

Tareas en los IDE

La mayor parte de los IDE permiten llevar a cabo las siguientes tareas:

- Ejecutar en modo depuración, para poder corregir los errores encontrados durante las pruebas.
- Analizar la consistencia y calidad del código fuente.
- Ejecución automática de pruebas.
- Control de versiones.
- Generación de documentación.
- Optimización del código (refactorización).

Hay *IDEs* creados para su uso con un único lenguaje de programación, pero también existen entornos de desarrollo multilenguaje.

Requisitos de JAVA



Para crear y ejecutar programas en Java, independientemente de que se use o no un IDE, es necesario instalar el equipo de desarrollo de Java, conocido de manera abreviada como **JDK** (*Java Development Kit*), en el que viene incorporado el entorno en tiempo de ejecución de Java, más conocido como **JRE** (*Java Runtime Environment*).

EL JDK contiene:

- **Un Compilador** (El "Traductor"): Es la herramienta principal (llamada ***javac***). Toma tu código fuente (los archivos ***.java*** que escribes) y lo traduce a un lenguaje intermedio llamado **Bytecode** (archivos ***.class***) que la máquina virtual de Java puede entender.
- **El JRE (*Java Runtime Environment*)**: Es el "**entorno de ejecución**". Esto es lo que la mayoría de la gente tiene instalado para usar aplicaciones de Java. A su vez, el JRE incluye la **JVM (*Java Virtual Machine*)**, que es el "motor" que finalmente ejecuta el Bytecode en cualquier sistema operativo.

Requisitos de JAVA

Resumiendo: piensa en el software de Java de esta manera:

- JVM (La Máquina Virtual): MV sobre la que se ejecuta el entorno de ejecución
- JRE (Entorno de Ejecución): Te permite conducir ejecutar programas en java, incluye el JVM.
- JDK (Kit de Desarrollo): Es el conjunto de herramientas. Te permite crear aplicaciones

Si solo quieres usar una aplicación Java, necesitas el JRE. Si quieres crear aplicaciones Java, necesitas el JDK (que ya incluye el JRE).

El JDK y el JRE de Java se instalan conjuntamente, puedes acceder a ellos en la siguiente web: [Java Downloads | Oracle](https://www.oracle.com/es/java/technologies/javase-downloads.html)



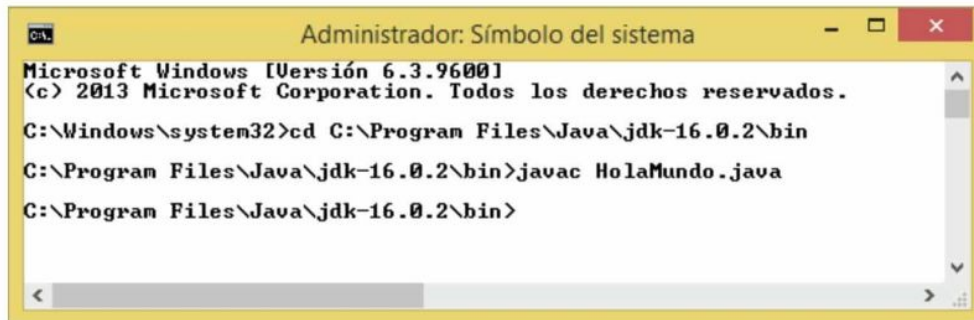
¡ Hola Mundo ! → Sin IDE

Cuando estén instalados el JDK y el JRE de Java, será posible crear programas en Java sin el empleo de ningún IDE.

Por ejemplo, veamos este código:

```
1 public class HolaMundo{  
2     public static void main (String[] args){  
3         System.out.println ("¡Hola, mundo!");  
4     }  
5 }
```

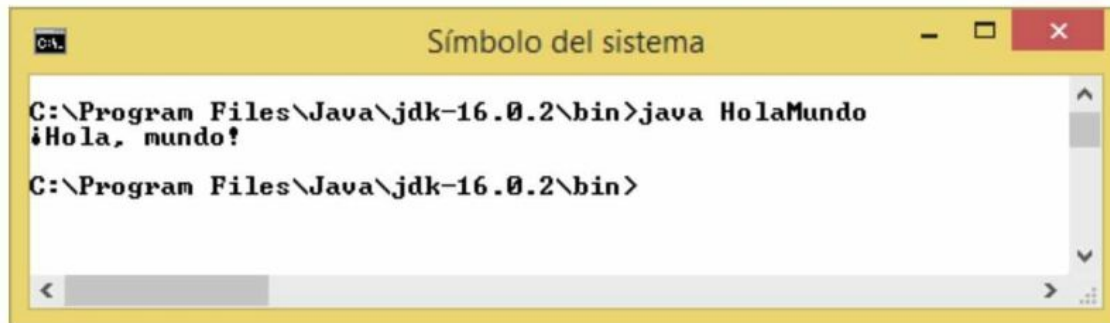
El siguiente paso es la compilación del programa, para lo que se usa el **compilador de Java: *javac*** ubicado en la carpeta correspondiente al JDK.



```
Administrador: Símbolo del sistema  
Microsoft Windows [Versión 6.3.9600]  
(c) 2013 Microsoft Corporation. Todos los derechos reservados.  
C:\Windows\system32>cd C:\Program Files\Java\jdk-16.0.2\bin  
C:\Program Files\Java\jdk-16.0.2\bin>javac HolaMundo.java  
C:\Program Files\Java\jdk-16.0.2\bin>
```

¡ Hola Mundo ! → Sin IDE

Tras compilar correctamente el programa, solo faltará ejecutarlo, para lo que, desde el símbolo del sistema, se escribe **java** y, después, el nombre del fichero sin extensión:



```
C:\Program Files\Java\jdk-16.0.2\bin>java HolaMundo
¡Hola, mundo!

C:\Program Files\Java\jdk-16.0.2\bin>
```

¿Qué te parece el proceso? ¿Te resulta sencillo? Imagina que tienes un código más complejo, con múltiples archivos y relaciones entre ellos.
¿Te parece este procedimiento viable?

2. Componentes de un entorno de desarrollo

Componentes de un entorno de desarrollo



Los componentes básicos de un IDE son:

- **Editor de textos:** se emplea para la escritura del código fuente del programa. Incluye las funciones propias de cualquier editor de textos (copiar, cortar, pegar, buscar, reemplazar, etc.) e incluye herramientas importantes para la escritura de los programas: marca en ciertos colores determinados elementos del lenguaje, como palabras reservadas y variables, resalta el inicio y el fin de cada bloque, etc. Esto contribuye a la legibilidad de los programas y es un apoyo relevante para las tareas de escritura y revisión del código fuente.
- **Compilador o intérprete:** comprueba la corrección del programa escrito en lenguaje fuente y, en caso de que sintácticamente sea correcto, genera el correspondiente código objeto o directamente el código ejecutable. Los compiladores traducen todo el programa fuente y almacenan el resultado, mientras que los intérpretes van traduciendo porciones de código y las van ejecutando, y no almacenan el resultado de la traducción.

Componentes de un entorno de desarrollo



- **Depurador (debugger):** localización y corrección de errores en el programa. Para ello, permite realizar tareas como:
 - Detención de la ejecución del programa en una determinada instrucción mediante el establecimiento de puntos de ruptura.
 - Ejecución del programa instrucción a instrucción.
 - Visualización de los valores de las variables a lo largo de la ejecución.

El depurador puede resultar muy útil para detectar y corregir determinados errores que, sin esta herramienta, resultaría complicado.

- **Constructor de interfaz gráfica:** permite crear aplicaciones con una interfaz gráfica de usuario (GUI, por sus siglas en inglés, *graphical user interface*). Una GUI se crea mediante la colocación de distintos controles en la pantalla, como botones, campos de texto, listas o barras de menús, con los cuales interactúa el usuario mediante el ratón o el teclado. La aplicación responde a las acciones (eventos) que realiza el usuario sobre estos controles. La mayoría de los entornos de desarrollo requieren la instalación de módulos adicionales para la creación de este tipo de aplicaciones.
- **Control de versiones:** controla los cambios que se realizan sobre los programas, creando diferentes versiones de estos a medida que se van incorporando dichos cambios.

3. Instalación

Instalación de NetBeans

La instalación de NetBeans requiere dos elementos:

- El JDK (Java Development Kit o Kit de Desarrollo de Java) es el paquete de software fundamental que necesitas para crear aplicaciones en Java. Contiene todo lo necesario no solo para ejecutar programas de Java, sino, lo más importante, para compilarlos y depurarlos.
- El propio software: IDE Netbeans

En la web de "Friends of Apache NetBeans" crea y ofrece los paquetes "todo-en-uno" que incluyen tanto el IDE de NetBeans como el JDK de Java en un solo instalador.



[Downloading Apache NetBeans 27](#)

[Friends of Apache NetBeans](#)

Crear un proyecto en NetBeans



Gestor de proyectos de NetBeans

Es el sistema interno que el IDE utiliza para entender, organizar y construir tu código.

Cuando creas un "Nuevo Proyecto" en NetBeans (Archivo > Nuevo Proyecto...), estás seleccionando un "gestor de proyectos" específico, que actúa como una plantilla. Este gestor se encarga de todo lo necesario para que tu aplicación funcione:

- **Organiza las carpetas:** Define una estructura estándar (dónde va tu código fuente src, dónde van las librerías lib, etc.).
- **Gestiona las dependencias:** Controla qué librerías externas (JARs u otros) necesita tu proyecto para funcionar.
- **Define cómo construir (Build):** Sabe cómo compilar tu código Java, empaquetarlo en un archivo .jar o .war y ejecutarlo.
- **Configura la depuración:** Prepara todo lo necesario para que puedas depurar tu código.

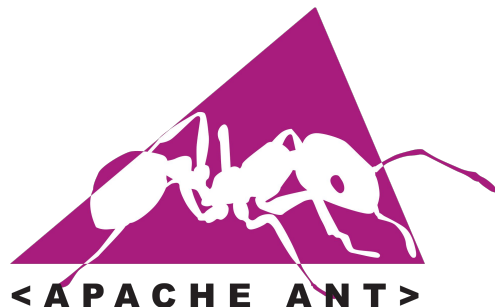
Crear un proyecto en NetBeans



NetBeans gestiona los proyectos principalmente a través de dos sistemas de construcción muy populares:

Ant (El gestor clásico de NetBeans):

- Este es el sistema de proyectos "nativo" de NetBeans para aplicaciones Java estándar.
- Cuando creas un proyecto "*Java with Ant*", NetBeans genera automáticamente unos scripts de construcción (un archivo *build.xml*) que le dicen a Ant cómo compilar, probar y ejecutar tu proyecto.
- Es el más sencillo si estás empezando y solo quieres crear una aplicación de escritorio o una librería simple.



Crear un proyecto en NetBeans



Maven (El gestor moderno y más potente):

- Maven es un estándar en la industria, no solo en NetBeans.
- Cuando creas un proyecto "*Java with Maven*", el gestor es Maven mismo. NetBeans simplemente actúa como una interfaz gráfica para él.
- Su poder reside en el archivo *pom.xml*, donde declaras las dependencias que necesitas (ej. "Quiero la librería X, versión 2.5"). Maven se encarga de descargarla automáticamente de internet y añadirla a tu proyecto.
- Es el gestor preferido para proyectos más grandes, aplicaciones web y cuando trabajas en equipo, ya que asegura que todos usen las mismas versiones de las librerías.

Además de estos, NetBeans también incluye gestores para otros tipos de proyectos, como Gradle (similar a Maven) o proyectos para PHP, C/C++ y JavaScript, cada uno con su propia lógica para construir y ejecutar ese tipo de aplicación.

Instalación de IntelliJ

La instalación de IntelliJ requiere dos elementos:

- El JDK (Java Development Kit o Kit de Desarrollo de Java) es el paquete de software fundamental que necesitas para crear aplicaciones en Java. Contiene todo lo necesario no solo para ejecutar programas de Java, sino, lo más importante, para compilarlos y depurarlos.
- El propio software: IntelliJ IDEA, usaremos la edición Community



[Descargar IntelliJ
IDEA](#)

Característica	Community (Gratis)	Ultimate (Pago)
Java y Kotlin	✓	✓
Maven / Gradle	✓	✓
Desarrollo Web (JavaScript, HTML)	✗ No	✓ Sí
Frameworks (Spring, Jakarta EE)	✗ No	✓ Sí
Bases de Datos (SQL)	✗ No	✓ Sí
Profilers de Rendimiento	✗ No	✓ Sí
Python, PHP, Go	✗ No	✓ Sí

Crear un proyecto en IntelliJ



IntelliJ IDEA gestiona los proyectos de una forma muy estructurada, centrada en dos conceptos clave: **Proyectos** y **Módulos**. Comprender esta diferencia es fundamental, ya que difiere de cómo lo hacen otros IDEs como NetBeans o Eclipse. Los conceptos principales a tener en cuenta son:

Proyecto (Project)

Es el contenedor de más alto nivel, como un espacio de trabajo. Contiene todos los módulos, la configuración del IDE y los ajustes de estilo de código.

Módulo (Module)

Esto es lo que otros IDEs (como NetBeans) llaman "proyecto". Es una unidad de código independiente (como una aplicación, una librería, etc.). Un proyecto puede tener múltiples módulos que dependen entre sí.

Crear un proyecto en IntelliJ



Sistema de Construcción (Build System)

IntelliJ te deja elegir cómo gestionar las dependencias y la compilación:

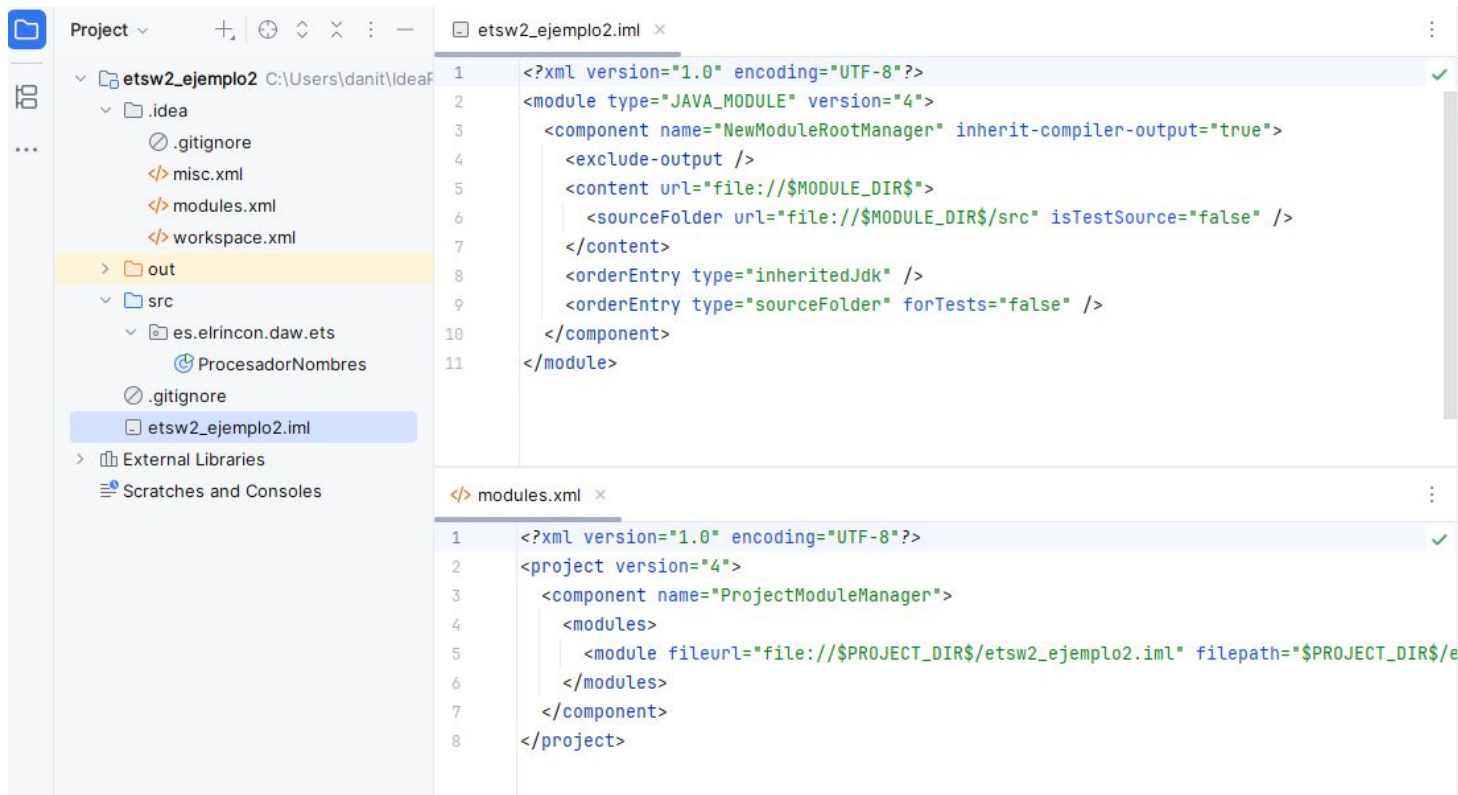
- **IntelliJ Nativo (JPS):** Es más simple. Las dependencias (archivos .jar) se deben añadir manualmente a través de la configuración del proyecto.
- **Maven/Gradle:** IntelliJ delega la gestión a estas herramientas. Lee los archivos *pom.xml* o *build.gradle* y automatiza las dependencias y la compilación.

Archivos de Configuración

IntelliJ guarda su configuración en dos lugares:

- **Carpeta .idea:** Contiene la configuración de tu Proyecto (tu espacio de trabajo).
- **Archivos .iml:** Se crea uno por cada Módulo y define sus carpetas de código y dependencias.

Crear un proyecto en IntelliJ



The screenshot displays the IntelliJ IDEA interface for a project named 'etsw2_ejemplo2'. The Project tool window on the left shows the file structure, including the 'src' directory and the 'etsw2_ejemplo2.iml' file. The main editor shows the content of the 'etsw2_ejemplo2.iml' and 'modules.xml' files.

etsw2_ejemplo2.iml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <module type="JAVA_MODULE" version="4">
3   <component name="NewModuleRootManager" inherit-compiler-output="true">
4     <exclude-output />
5     <content url="file://$MODULE_DIR$">
6       <sourceFolder url="file://$MODULE_DIR$/src" isTestSource="false" />
7     </content>
8     <orderEntry type="inheritedJdk" />
9     <orderEntry type="sourceFolder" forTests="false" />
10  </component>
11 </module>
```

modules.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project version="4">
3   <component name="ProjectModuleManager">
4     <modules>
5       <module fileurl="file://$PROJECT_DIR$/etsw2_ejemplo2.iml" filepath="$PROJECT_DIR$/e
6     </modules>
7   </component>
8 </project>
```

Crear un proyecto en IntelliJ

IntelliJ gestiona los proyectos principalmente a través de varios sistemas de construcción:

Maven



- El estándar de la industria. Usa un archivo ***pom.xml*** (XML).
- Dependencias automáticas. Solo declaras el nombre de una librería y Maven la descarga.
- Ideal para proyectos empresariales, estándar y estables.

Gradle



- El constructor moderno y rápido. Usa un archivo ***build.gradle*** (código).
- Dependencias automáticas. Es más rápido y flexible que Maven.
- Ideal para proyectos de Android, microservicios y builds complejos.

IntelliJ (JPS)

- El constructor nativo y simple del IDE. No usa un archivo de build especial.
- Dependencias manuales. Tienes que descargar los archivos ***.jar*** a mano y añadirlos al proyecto.
- Ideal para proyectos muy pequeños, aprender Java o prototipos rápidos sin librerías externas.

Crear un proyecto en IntelliJ



Los pasos esenciales para crear un proyecto en IntelliJ:

1. Inicia el Asistente:
 - a. Abre IntelliJ y haz clic en "New Project" (Nuevo Proyecto) en la pantalla de bienvenida.
 - b. O, si ya estás dentro, ve a File > New > Project....
2. Configura el Proyecto:
 - a. **Name:** Dale un nombre a tu proyecto (ej. MiApp).
 - b. **Build System:** Selecciona "IntelliJ" (es el más simple para empezar).
 - c. **JDK:** Asegúrate de que esté seleccionado tu JDK (Java Development Kit). Si no, selecciónalo de la lista.
 - d. **Add sample code:** Marca esta casilla. Te creará un archivo Main.java con un "Hola Mundo" listo.
3. Crea y Ejecuta:
 - a. Haz clic en "Create".
 - b. Abre el archivo src/Main.java que se creó.
 - c. Haz clic en el icono de Play verde junto al método main para ejecutar tu programa.

4. Depuración de Java

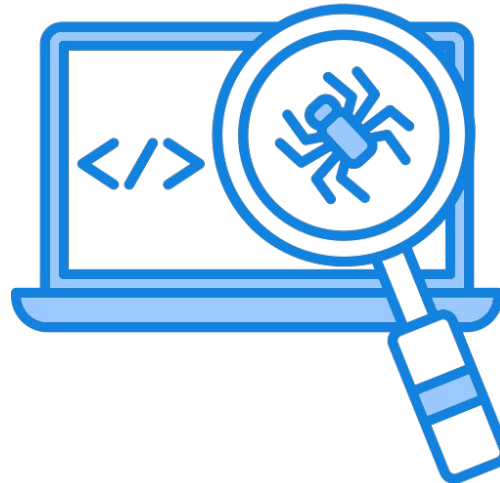
Depuración

Definición

La **depuración** (o **debugging**) es el proceso de **encontrar y corregir errores** (conocidos como **bugs** o "bichos") en el código de un programa de software. En términos sencillos, es el trabajo de detective que haces cuando tu programa no se comporta como esperabas.

El objetivo no es solo ver qué falla (como un crash o un resultado incorrecto), sino entender por qué falla, para poder solucionarlo de raíz.

DEBUG 🐛 en Java ☕ con Netbeans 👤 🖥️
Ejecuta tu CÓDIGO LÍNEA a LÍNEA 🐾



¿Qué implica depurar?

Cuando depuras, no solo lees el código, sino que lo ejecutas de forma controlada usando herramientas especiales (un depurador o debugger). Esto te permite:

- **Pausar la Ejecución:** Puedes poner "**paradas**" llamadas puntos de ruptura (**breakpoints**) para detener el programa en una línea específica.
- **Ir Paso a Paso:** Una vez pausado, puedes ejecutar el programa línea por línea.
- **Inspeccionar Variables:** Mientras está pausado, puedes mirar el valor de todas tus variables en ese preciso momento, como si tomaras una radiografía del programa.

El objetivo siempre será encontrar la causa por la que el programa tiene un comportamiento inesperado. Al ver cómo cambian los valores, puedes detectar el momento exacto en que algo sale mal y entender la lógica defectuosa.



Depuración de Java en NetBeans

Depuración de Java en NetBeans



Depurar en NetBeans es un proceso visual e intuitivo. Los conceptos clave son los **puntos de ruptura (breakpoints)**, la **ejecución paso a paso** y la **inspección de variables**.

Pasos para realizar la depuración:

1. Poner Puntos de Ruptura (Breakpoints)

Un punto de ruptura es una señal que le dice al depurador que pause la ejecución del programa en una línea específica, justo antes de que se ejecute.

¿Cómo poner uno?

Haz clic en el margen izquierdo del editor de código, junto al número de la línea donde quieres que el programa se detenga. Aparecerá un cuadrado rojo para indicar el breakpoint.

¿Dónde ponerlos?

Colócalos en lugares lógicos donde sospeches que hay un problema o donde quieras empezar a analizar el estado de tu programa.



```
26  */
27  public static long calcularFactorial(int n) {
28      // Sugerencia: Pon un punto de ruptura en la siguiente línea.
29      long resultado = 0; // ¿Debería empezar en 0 al multiplicar? Mmm...
30
31      // Sugerencia: Pon otro punto de ruptura aquí para ver cómo cambia 'i'.
32      for (int i = 1; i < n; i++) { // ¿La condición i < n es correcta?
33          resultado = resultado * i;
34      }
35
36      return resultado;
37  }
38  }
39  }
```

Depuración de Java en NetBeans



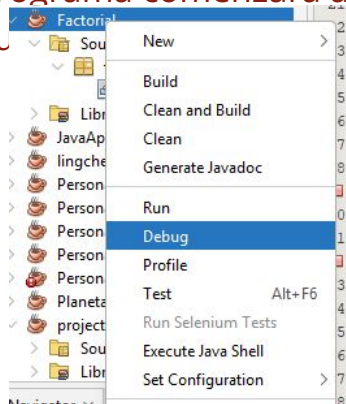
Pasos para realizar la depuración:

2. Iniciar la Sesión de Depuración

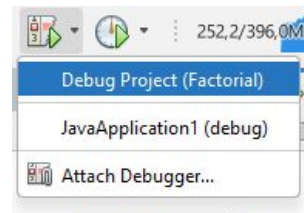
En lugar de ejecutar el programa de la forma normal, debes iniciarlo en **modo de depuración**.

- Haz clic derecho en el archivo que contiene tu método main. En el menú contextual, selecciona **Debug File** (Depurar Archivo).
- También puedes usar el atajo de teclado **Ctrl + F5** o ir al menú **Debug > Debug File**.

El programa comenzará a ejecutarse y se detendrá automáticamente en el primer punto de ruptura que encuentres. El programa está pausado ahí.



```
2  * Calcula el factorial de un número dado.  
3  * CONTIENE UN ERROR LÓGICO.  
4  * @param n El número entero positivo.  
5  * @return El factorial del número n.  
6  */  
7  public static long calcularFactorial(int n) {  
8      // Sugerencia: Pon un punto de ruptura en la siguiente  
9      long resultado = 0; // ¿Debería empezar en 0 al multipl  
10  
11     // Sugerencia: Pon otro punto de ruptura aquí para ver  
12     for (int i = 1; i < n; i++) { // ¿La condición i < n es  
13         resultado = resultado * i;  
14     }  
15  
16     return resultado;  
17 }  
18
```



Depuración de Java en NetBeans



Pasos para realizar la depuración:

3. Controlar la Ejecución (Navegación Paso a Paso)

Una vez que el programa está pausado, puedes controlar cómo avanza. Para esto, usarás la barra de herramientas de depuración que aparece en la parte superior. Estos son los controles más importantes:

- **Continuar (F5):** Reanuda la ejecución normal del programa hasta que encuentre el siguiente punto de ruptura o hasta que el programa termine.
- **Saltar (Step Over - F8):** Ejecuta la línea actual resaltada y se detiene en la siguiente línea del mismo método. Si la línea actual contiene una llamada a otro método, lo ejecutará por completo sin entrar en él. Es el control más usado.
- **Entrar (Step Into - F7):** Si la línea actual es una llamada a un método, este comando entra dentro de ese método y se detiene en su primera línea. Es útil para depurar el código de otros métodos que tú has escrito.
- **Salir (Step Out - Ctrl + F7):** Si has entrado en un método con "Step Into", este comando ejecutará el resto de ese método y te devolverá a la línea donde fue llamado.
- **Finalizar Sesión (Shift + F5):** Detiene la depuración por completo.

Depuración de Java en NetBeans



Pasos para realizar la depuración:

4. Inspeccionar Variables y Expresiones

El principal objetivo de la depuración es **ver qué valores tienen tus variables en un momento determinado**.

- **Ventana de Variables:** Cuando la ejecución está pausada, en la parte inferior de NetBeans se abrirá la pestaña "**Variables**" (o ve a **Window > Debugging > Variables**). Aquí podrás ver una lista de todas las variables locales y sus valores actuales. Los valores que acaban de cambiar se resaltan en azul.
- **Tooltips (Información emergente):** Simplemente pasa el cursor del ratón sobre una variable directamente en el editor de código. Aparecerá una pequeña ventana emergente mostrando su valor actual.
- **Watches (Inspecciones):** Puedes "vigilar" variables o expresiones específicas.
 - Haz clic derecho en una variable y selecciona "**New Watch...**".
 - Se abrirá la ventana "**Watches**" donde podrás ver el valor de esa variable durante toda la sesión de depuración. También puedes añadir expresiones complejas (ej. `contador * 2 + offset`).

Depuración de Java en NetBeans



Pasos para realizar la depuración:

5. La Pila de Llamadas (Call Stack)

La ventana "**Call Stack**" es extremadamente útil. Te muestra la secuencia de llamadas a métodos que te llevaron al punto actual de ejecución.

- La línea de arriba es el método actual.
- Las líneas de abajo son los métodos que lo llamaron.
- Puedes hacer doble clic en cualquier método de la pila para saltar a la línea de código donde se realizó la llamada y ver el estado de las variables en ese contexto.
- Esto es fundamental para entender cómo has llegado a un punto problemático en tu código.

Depuración de Java en NetBeans



Pasos para realizar la depuración:

Otros aspectos de interés: Breakpoints Condicionales

A veces quieres que un breakpoint solo se active si se cumple una cierta condición (por ejemplo, en un bucle, cuando la variable `i` valga 50).

- Haz clic derecho en el icono del breakpoint (el cuadrado rojo).
- Selecciona Breakpoint > Properties.
- En la ventana que aparece, puedes establecer una Condición. Por ejemplo: `i == 50`.
- Ahora, el depurador solo se detendrá en esa línea si la condición es verdadera.



IntelliJ IDEA
JETBRAINS IDE

Depuración de Java en IntelliJ IDEA

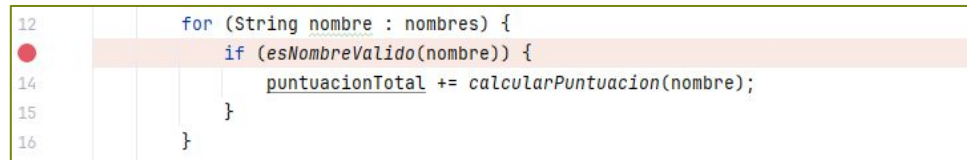
Depuración de Java en IntelliJ



IntelliJ IDEA tiene un depurador visual muy potente. El proceso se basa en **puntos de ruptura (breakpoints)**, la **ejecución paso a paso** y la **inspección de variables**.

Pasos para realizar la depuración:

1. Poner Puntos de Ruptura (Breakpoints)



Un punto de ruptura es una señal que le dice al depurador que pause la ejecución del programa en una línea específica, justo antes de que se ejecute.

¿Cómo poner uno?

Haz clic en el margen izquierdo del editor de código (conocido como gutter), junto al número de la línea donde quieres que el programa se detenga. Aparecerá un círculo rojo para indicar el breakpoint.

¿Dónde ponerlos?

Colócalos en lugares lógicos donde sospeches que hay un problema o donde quieras empezar a analizar el estado de tu programa, un método que sospeches que falla, o justo antes de una operación crítica.

Depuración de Java en IntelliJ



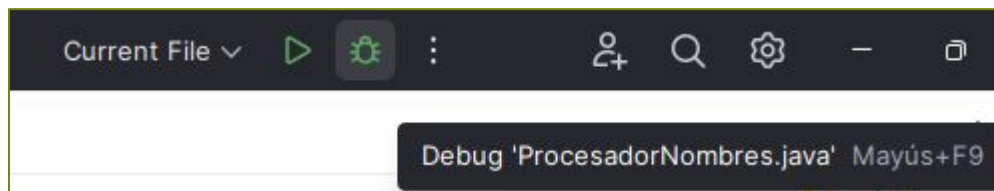
Pasos para realizar la depuración:

2. Iniciar la Sesión de Depuración

En lugar de ejecutar el programa de la forma normal, debes iniciarlo en **modo de depuración**.

- Busca el **icono del bicho verde** en la barra de herramientas (o junto a tu método main).
- Haz clic en él para **"Debug"** (Depurar).
- También puedes usar el atajo de teclado **Shift + F9**.

El programa comenzará y se detendrá automáticamente en el primer punto de ruptura que encuentre. La línea se **resaltará en azul**, y en la parte inferior de la pantalla aparecerá la ventana de herramientas **"Debug"**.



Pasos para realizar la depuración:

3. Controlar la Ejecución (Navegación Paso a Paso)

Una vez que el programa está pausado, la ventana "Debug" te muestra los controles para avanzar. Estos son los botones más importantes:

- **Resume Program (F9):** Reanuda la ejecución normal del programa. Correrá hasta que encuentre el siguiente punto de ruptura o hasta que termine.
- **Saltar (Step Over - F8):** Ejecuta la línea actual resaltada y se detiene en la siguiente línea del mismo método. Si la línea actual contiene una llamada a otro método, lo ejecutará por completo sin entrar en él. Es el control más usado.
- **Entrar (Step Into - F7):** Si la línea actual es una llamada a un método, este comando entra dentro de ese método y se detiene en su primera línea. Es útil para depurar el código de otros métodos que tú has escrito.
- **Salir (Step Out - Shift + F8):** Si has entrado en un método con "Step Into", este comando ejecutará el resto de ese método y te devolverá a la línea donde fue llamado.
- **Finalizar Sesión (Ctrl + F2):** Detiene la depuración por completo.

Depuración de Java en IntelliJ



Pasos para realizar la depuración:

3. Controlar la Ejecución (Navegación Paso a Paso)

Una vez que el programa está pausado, la ventana "Debug" te muestra los controles para avanzar. Estos son los botones más importantes:



Depuración de Java en IntelliJ



Pasos para realizar la depuración:

4. Inspeccionar Variables y Expresiones

El principal objetivo de la depuración es **ver qué valores tienen tus variables en un momento determinado**.

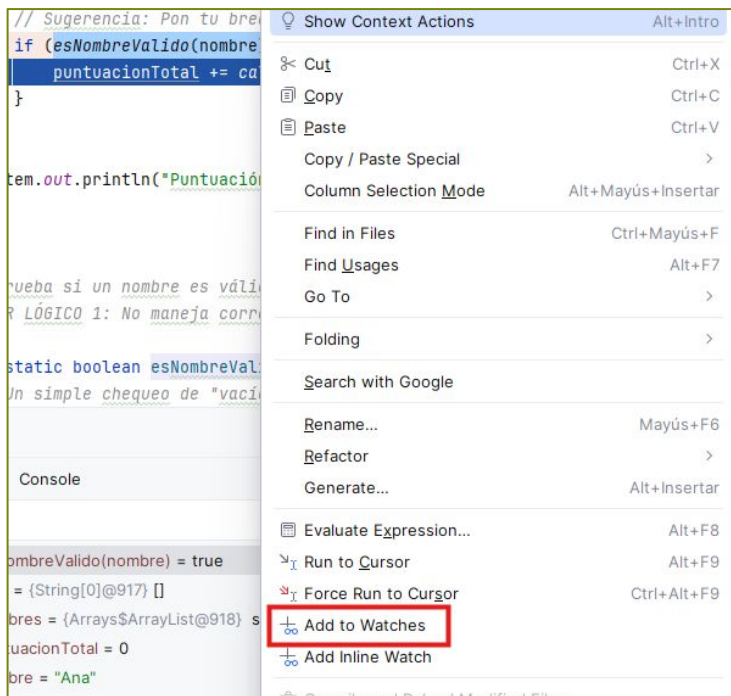
- **Panel de Variables:** En la ventana "**Debug**" (abajo), la pestaña "**Variables**" es la más importante. Te muestra todas las variables locales y sus valores actuales. A medida que avanzas con F8, verás cómo estos valores cambian (IntelliJ los resalta en color).
- **Valores en el Editor:** Simplemente pasa el cursor del ratón sobre una variable directamente en el editor de código. Aparecerá una pequeña ventana emergente mostrando su valor actual.
- **Watches (Inspecciones):** Puedes hacer clic derecho en una variable y seleccionar "**Add to Watches**". Esto la fija en un panel separado para que puedas vigilarla fácilmente, incluso si no está en el ámbito actual. Además es posible añadir expresiones complejas. (ej. `contador * 2 + offset`).

Depuración de Java en IntelliJ



Pasos para realizar la depuración:

4. Inspeccionar Variables y Expresiones



Depuración de Java en IntelliJ



Pasos para realizar la depuración:

5. La Pila de Llamadas (Call Stack)

La ventana "**Call Stack**" es extremadamente útil. Te muestra la secuencia de llamadas a métodos que te llevaron al punto actual de ejecución.

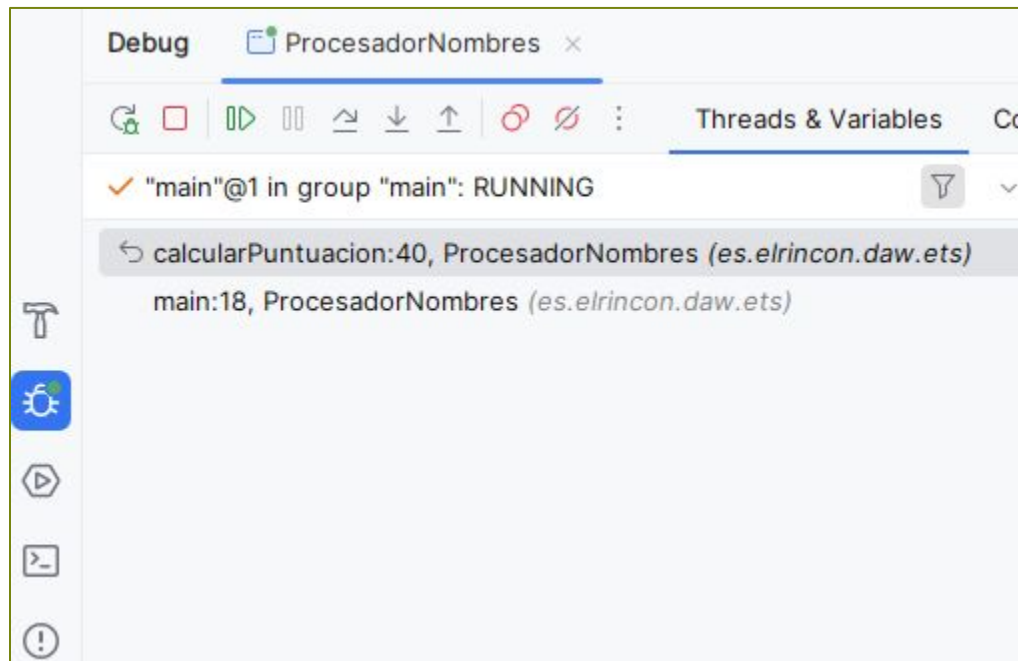
- En la ventana "**Debug**", la pestaña "**Frames**" (o "Call Stack") te muestra la lista de métodos que se han llamado.
- El método de arriba es donde estás ahora.
- El método de abajo es el que lo llamó, y así sucesivamente (usualmente terminando en **main**).
- Puedes hacer clic en cualquier método de la pila para "viajar en el tiempo" y ver el estado de las variables en el momento en que se hizo esa llamada.

Depuración de Java en IntelliJ



Pasos para realizar la depuración:

5. La Pila de Llamadas (Call Stack)



Depuración de Java en IntelliJ



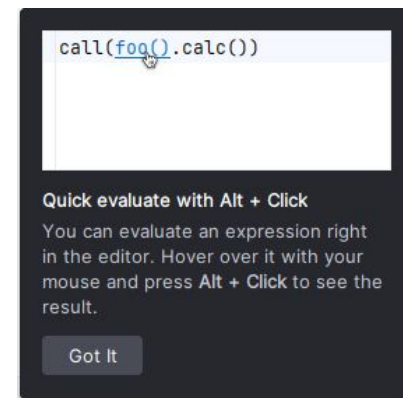
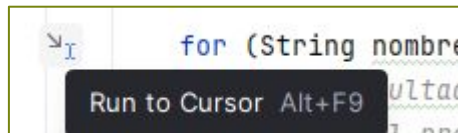
Pasos para realizar la depuración:

Otros aspectos de interés: Breakpoints Condicionales

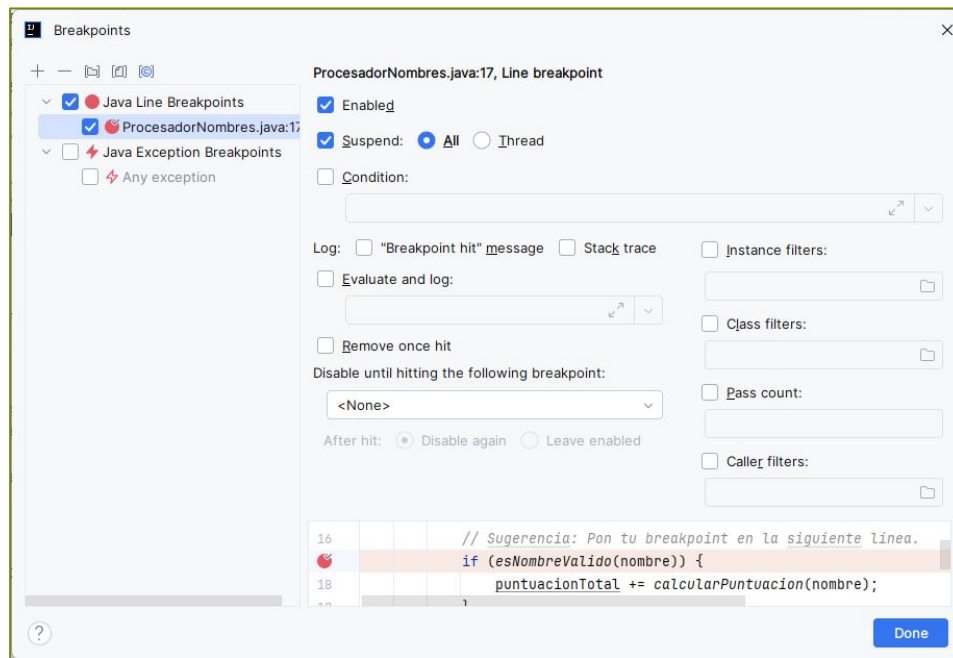
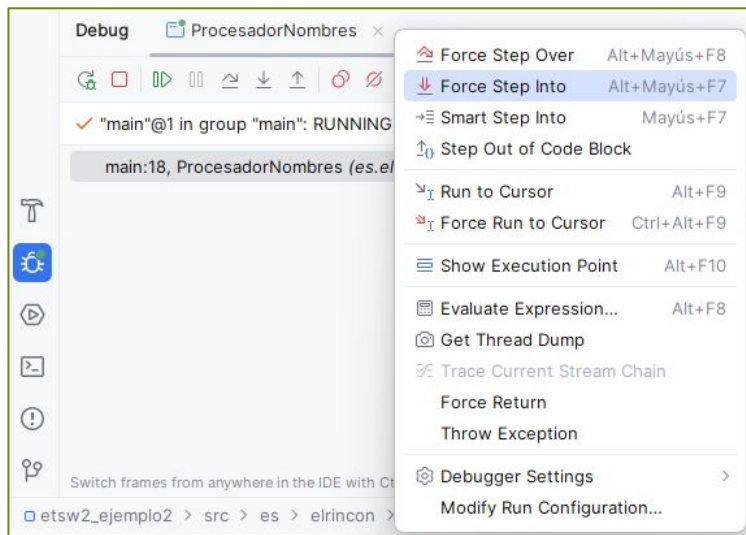
Si tienes un error dentro de un bucle que se repite 1000 veces, pero solo falla cuando *i* vale 500, no querrás presionar F8 500 veces.

- Haz clic derecho sobre el círculo rojo del breakpoint.
- Aparecerá un campo llamado "Condition" (Condición).
- Escribe una expresión Java, por ejemplo: `i == 500`.

Ahora, el depurador solo se detendrá en esa línea si la condición es verdadera.



Depuración de Java en IntelliJ



Depuración de Java en IntelliJ - ejemplos

Ejemplo 1

Si ejecutas este programa, la salida será:

El factorial de 5 es: 0

El resultado correcto debería ser 120,

```
public class CalculadoraFactorial {  
  
    public static void main(String[] args) {  
        int numero = 5;  
        long resultadoFactorial = calcularFactorial(numero);  
  
        // El resultado debería ser 120, pero el programa mostrará 0.  
        System.out.println("El factorial de " + numero + " es: " + resultadoFactorial);  
    }  
  
    /**  
     * Calcula el factorial de un número dado.  
     * CONTIENE UN ERROR LÓGICO.  
     * @param n El número entero positivo.  
     * @return El factorial del número n.  
     */  
    public static long calcularFactorial(int n) {  
        // Sugerencia: Pon un punto de ruptura en la siguiente línea.  
        long resultado = 0;  
  
        // Sugerencia: Pon otro punto de ruptura aquí para ver cómo cambia 'i'.  
        for (int i = 1; i < n; i++) {  
            resultado = resultado * i;  
        }  
  
        return resultado;  
    }  
}
```

Depuración de Java en IntelliJ - ejemplos



Ejemplo 2

Si ejecutas este programa, la salida será:

Puntuación total calculada: 18

El resultado correcto debería ser 16.

```
import java.util.Arrays;
import java.util.List;

public class ProcesadorNombres {

    public static void main(String[] args) {
        List<String> nombres = Arrays.asList("Ana", "Luis", "", "Carlos", " EVA ");
        int puntuacionTotal = 0;

        for (String nombre : nombres) {
            // El resultado esperado para la lista es: 3 (Ana) + 4 (Luis) + 6 (Carlos) + 3 (EVA) = 16
            // Pero el programa dará 18... ¿Por qué?

            // Sugerencia: Pon tu breakpoint en la siguiente línea.
            if (esNombreValido(nombre)) {
                puntuacionTotal += calcularPuntuacion(nombre);
            }
        }

        System.out.println("Puntuación total calculada: " + puntuacionTotal);
    }

    // CONTINÚA EN LA SIGUIENTE DIAPOSITIVA ##### ->
```


Depuración de Java en IntelliJ - ejemplos

Ejemplo 2

Si ejecutas este programa, la salida será:

Puntuación total calculada: 18

El resultado correcto debería ser 16.

```
// → CONTINUACIÓN #####

/**
 * Comprueba si un nombre es válido.
 */
public static boolean esNombreValido(String nombre) {
    // Un simple chequeo de "vacío" no es suficiente.
    return !nombre.isEmpty();
}

/**
 * Calcula la puntuación de un nombre.
 */
public static int calcularPuntuacion(String nombre) {
    // Debería medir el nombre "limpio" (sin espacios extra).
    return nombre.length();
}
}
```

CONTINUARÁ...

